

Cloud Security Study Plan — Phase 5 Completion Log

GitHub Actions CI/CD + Terraform IaC Hardening (Weeks 19-22)

Part 1 — GitHub Actions CI/CD (Weeks 19-20)

1.1 Repository Structure

Existing documentation repo (CloudKomputing) was kept as-is on main. A new branch, docker-cicd, was created to hold purely infrastructure code — Terraform config and the Flask app — separated from the PDFs/notes living on main.

```
git checkout -b docker-cicd
git add .gitignore terraform-docker-host flask-app
git commit -m "Add Terraform infra and Flask app for Docker CI/CD study"
git push -u origin docker-cicd
```

Files inherited from main (PDFs, README, notes) were subsequently removed from docker-cicd only, keeping that branch as pure infra code:

```
git rm "AWS Cloud Practitioner Completion.pdf" "AWS Cloud Quest - Cloud Practitioner
Completion.pdf" \
  "Docker + Terraform.pdf" "OverTheWire.txt" "OvertheWire
Linux_SSH_Command_Reference_Full.pdf" \
  README.md "aws-cloud-quest-cloud-practitioner-training-badge.png"
flaws2_cloud_journey.pdf flaws_cloud_journey.pdf
git commit -m "Remove non-infra files from docker-cicd branch"
git push
```

1.2 Retrieving Files Created Directly on EC2

app.py and Dockerfile had originally been created via SSH directly on the EC2 instance, not on the local machine. Used scp to pull them down into the local repo rather than retyping:

```
scp -i "terraform-docker-host\docker-study-key-my5.pem"
ec2-user@<EC2_IP>:~/flask-app/app.py flask-app\app.py
scp -i "terraform-docker-host\docker-study-key-my5.pem"
ec2-user@<EC2_IP>:~/flask-app/Dockerfile flask-app\Dockerfile
```

1.3 Authentication Strategy for the Pipeline

Considered three options for how GitHub Actions would reach the EC2 instance for deployment:

- Widen the security group to GitHub's published IP ranges — rejected: hundreds of CIDR blocks, changes frequently, still overly broad
- Open SSH to 0.0.0.0/0 — rejected: permanent internet-facing SSH exposure, unacceptable for a security-focused project
- Chosen: dynamic, temporary IP allowlisting — the workflow fetches its own runner's public IP at the start of each run, adds a single-IP security group rule, deploys, then revokes that exact rule at the end (using `if: always()` so cleanup runs even on failure)

1.4 IAM User for GitHub Actions

Created a scoped IAM user (github-actions-deploy) with an inline least-privilege policy — not an admin user — limited to exactly the actions the pipeline needs:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "ec2:AuthorizeSecurityGroupIngress",
      "ec2:RevokeSecurityGroupIngress",
      "ec2:DescribeSecurityGroups",
      "ec2:DescribeInstances"
    ],
    "Resource": "*"
  }]
}
```

1.5 GitHub Secrets Configured

- AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY — from the github-actions-deploy IAM user
- AWS_REGION — ap-southeast-5
- EC2_INSTANCE_ID — kept in sync manually whenever Terraform replaces the instance
- EC2_SG_ID — the security group ID (sg-0c5aa381003f94ef0)
- EC2_SSH_KEY — full contents of the .pem private key
- EC2_USER — ec2-user

1.6 Workflow File (.github/workflows/deploy.yml)

Triggers on push to docker-cicd, scoped to changes under flask-app/. Steps: checkout, configure AWS credentials, fetch runner IP, authorize that IP for SSH, look up the EC2 instance's current public IP, copy app files via SCP, rebuild and restart the Docker container over SSH, then revoke the temporary SSH rule.

```
on:
  push:
    branches: [docker-cicd]
    paths:
      - "flask-app/**"
```

Key steps used community actions aws-actions/configure-aws-credentials, appleboy/ssh-action, and appleboy/scp-action.

1.7 First Successful Run

Result: Success, 58 seconds total duration. Confirmed live by loading the app in a browser and seeing the updated message text, proving the pipeline deployed real code changes rather than just completing without effect.

1.8 Recovery Incident — AMI Drift Forced Instance Replacement

During later work, terraform apply unexpectedly destroyed and recreated the EC2 instance. Root cause: the aws_ami data source uses most_recent = true, and AWS had published a newer Amazon Linux 2023

AMI since the last apply, so Terraform detected a diff and replaced the instance — wiping Docker and the Flask app files from the old instance's disk.

Fix applied to ec2.tf to prevent recurrence:

```
lifecycle {  
  ignore_changes = [ami]  
}
```

Recovery process: reinstalled Docker manually, recreated app.py/Dockerfile from the conversation history, rebuilt and re-ran the container, then re-pulled the files down locally via scp. This also demonstrated why Terraform automatically updates dependent resources — the Lambda's `INSTANCE_ID` environment variable updated itself to the new instance ID with no manual editing needed, since it referenced `aws_instance.docker_host.id` directly.

Part 2 — IaC Security Scanning (Weeks 21-22)

2.1 Tool Selection: Trivy instead of tfsec

The original study plan specified tfsec. Research confirmed tfsec is deprecated — Aqua Security merged its entire ruleset into Trivy in 2024, and tfsec has received no meaningful updates since. All tfsec check IDs work unchanged in Trivy, which also covers containers, Kubernetes, and CloudFormation in the same tool. Trivy was used instead as the current industry-standard equivalent.

```
winget install --id=AquaSecurity.Trivy -e  
trivy --version
```

2.2 Initial Scan Results

Ran `trivy config .` against `terraform-docker-host`. Found 4 misconfigurations across 3 files:

- AWS-0028 (HIGH) — EC2 instance does not require IMDSv2 token
- AWS-0131 (HIGH) — Root EBS volume not encrypted
- AWS-0066 (LOW) — Lambda function has no X-Ray tracing
- AWS-0104 (CRITICAL) — Security group allows unrestricted egress (0.0.0.0/0)

2.3 Fixes Applied

IMDSv2 enforcement and root volume encryption were fixed in `ec2.tf`:

```
root_block_device {  
  volume_size = 30  
  volume_type = "gp3"  
  encrypted   = true  
}  
  
metadata_options {  
  http_tokens = "required"  
  http_endpoint = "enabled"  
}
```

Note: changing `encrypted` on an existing root volume forces instance replacement (cannot be applied in-place) — same recovery cycle as the AMI drift incident was repeated here intentionally.

2.4 Auto-Bootstrapping Docker via user_data

Since instance replacement events (AMI changes, encryption changes, future config changes) will keep happening, added a user_data script so Docker installs itself automatically on every first boot, removing the need to ever manually reinstall Docker again after a replacement:

```
user_data = <<-EOF
#!/bin/bash
dnf update -y
dnf install -y docker
systemctl enable --now docker
usermod -aG docker ec2-user
EOF
```

Verified on the next instance replacement: docker --version returned a valid version immediately after boot, with zero manual installation steps.

2.5 Findings Consciously Accepted (Not Fixed)

Documented in SECURITY_ACCEPTED_FINDINGS.md, committed to the repo:

- AWS-0066 (LOW, X-Ray tracing) — accepted; the Lambda is a simple 3-line runtime check with no complex call chain, and CloudWatch Logs already provide sufficient visibility for its scope
- AWS-0104 (CRITICAL, unrestricted egress) — accepted; the host needs outbound access to Docker Hub, dnf mirrors, and GitHub, which use dynamic IP ranges impractical to allowlist. Ingress remains tightly restricted to the caller's current IP. Documented as a defensible exception for a dev/build host, distinct from production workloads handling sensitive data.

2.6 Final Scan State

Re-ran trivy config . after fixes: ec2.tf returned 0 findings. Only the two consciously accepted findings remain, both documented with written justification rather than left as unexplained scanner noise.

2.7 Status

Phase 5 is now fully complete: Docker (Weeks 17-18), GitHub Actions CI/CD (Weeks 19-20), and Terraform + IaC security scanning (Weeks 21-22). The infrastructure is self-healing — instance replacement events no longer require manual Docker reinstallation, dependent resources (Lambda env vars) update automatically, and every remaining scan finding is either fixed or explicitly risk-accepted with reasoning.

Next: Phase 6 — CloudGoat scenarios (Weeks 23-24) and HackTheBox Cloud Exploitation track (Weeks 25-26, budget dependent).